EXP 7 : To implement Banker's Algorithm to find the safe state for deadlock.

```
lab-03-17@lab-03-17-OptiPlex-3280-AIO: ~
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ gedit bankers_algo_1290.c
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ gcc bankers_algo_1290.c
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ ./a.out
Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

System is in SAFE state.
Safe Sequence: P1 P3 P4 P0 P2
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$
```

```
Open                                bankers_algo_1290.c
#include <stdio.h>

int main() {
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];

    // Input Allocation Matrix
    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }

    // Input Maximum Matrix
    printf("\nEnter Maximum Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    // Input Available Resources
    printf("\nEnter Available Resources:\n");
    for(i = 0; i < m; i++) {
        scanf("%d", &avail[i]);
    }

    // Calculate Need Matrix
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
```

Open

bankers_algo_1290.c

```c
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    int finish[n], safeSeq[n];
    for(i = 0; i < n; i++) {
        finish[i] = 0;
    }

    int count = 0;

    while(count < n) {
        int found = 0;

        for(i = 0; i < n; i++) {
            if(finish[i] == 0) {

                // Check if need <= avail
                int flag = 1;
                for(j = 0; j < m; j++) {
                    if(need[i][j] > avail[j]) {
                        flag = 0;
                        break;
                    }
                }

                if(flag) {
                    // Add allocation to available
                    for(k = 0; k < m; k++) {
                        avail[k] += alloc[i][k];
                    }

                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0) {
            printf("\nSystem is NOT in safe state!\n");
            return 0;
        }
    }
```